Assignment No: 12

Title: Develop a PHP application demonstrating form handling, validation, string manipulation, and session management.

Aim: To develop a server-side PHP application that receives data from an HTML form, validates and sanitizes it on the server, applies PHP string-manipulation functions, and maintains user state across requests using sessions.

Objectives:

1. To understand the request-response cycle of a server-side scripting language.
2. To handle form data submitted by the GET and POST methods using $_GET and $_POST.
3. To validate and sanitize user input on the server using filter_var and regular expressions.
4. To implement the sticky-form pattern so that valid values survive a failed submission.
5. To apply PHP string functions such as strlen, strtoupper, ucwords, str_replace, substr, trim, explode, implode and str_word_count.
6. To start, read, modify and destroy a session using $_SESSION.
7. To protect a page so that it can only be opened by a logged-in user.

Hardware Used:

- Computer or Laptop
- Keyboard
- Mouse
- Minimum 4 GB RAM

Software Used:

- Visual Studio Code (VS Code)
- Web Browser (Google Chrome, Microsoft Edge, or Firefox)
- Operating System: Windows / Linux / macOS
- XAMPP / WAMP / LAMP stack (Apache + PHP 8)
- PHP 8.0 or later

Theory:

1. Introduction to PHP

PHP (Hypertext Preprocessor) is a server-side scripting language. The PHP code inside a .php file is executed by the web server, and only the resulting HTML is sent to the browser, so the source is never visible to the client. PHP code is written between <?php and ?> tags and every statement ends with a semicolon.

2. Form Handling

An HTML form sends data to a PHP script named in its action attribute, using the verb in its method attribute. PHP exposes the values in superglobal arrays, keyed by the name attribute of each control.

Superglobal	Contains

$_GET	Values sent in the query string; visible in the URL, limited in size.
$_POST	Values sent in the request body; not visible, no practical size limit.
$_REQUEST	The union of $_GET, $_POST and $_COOKIE.
$_FILES	Details of uploaded files.
$_SESSION	Per-user data persisted on the server between requests.
$_SERVER	Request metadata such as REQUEST_METHOD and PHP_SELF.

A form should post to itself and the script should branch on $_SERVER["REQUEST_METHOD"] === "POST", so that the same file renders the empty form, the errors and the result.

3. Validation and Sanitization

Validation asks whether a value is acceptable; sanitization removes or escapes characters that could be dangerous. Client-side validation can be bypassed entirely, so server-side validation is mandatory.

Function	**Purpose**
trim($v)	Removes leading and trailing whitespace
htmlspecialchars($v)	Escapes < > & " to prevent XSS when echoing
stripslashes($v)	Removes backslashes added by old magic quotes
filter_var($v, FILTER_VALIDATE_EMAIL)	Returns the email or false
filter_var($v, FILTER_VALIDATE_INT, ...)	Validates an integer within a range
preg_match($pattern, $v)	Tests the value against a regular expression
empty($v) / isset($v)	Tests for an empty or undefined value

4. String Manipulation Functions

Function	**Returns**
strlen($s)	Number of bytes in the string
mb_strlen($s)	Number of characters, multibyte safe (needed for Devanagari)
strtoupper / strtolower	The string in upper or lower case
ucfirst / ucwords	First letter, or every word's first letter, capitalised
str_replace($a, $b, $s)	The string with every $a replaced by $b
substr($s, $start, $len)	A slice of the string
strpos($s, $needle)	Index of the first occurrence, or false
explode($sep, $s) / implode	String to array, and array back to string
str_word_count($s)	Number of words

strrev($s)	The reversed string
sprintf / number_format	Formatted output

5. Session Management

HTTP is stateless: each request is independent. A session gives the browser a session id in a cookie and stores the associated data on the server, so values survive from one request to the next.

Call	Effect
session_start()	Starts or resumes the session; must run before any output
$_SESSION['k'] = $v	Stores a value for this visitor
isset($_SESSION['k'])	Checks whether a value exists (used for login guards)
session_regenerate_id(true)	Issues a new id after login, preventing fixation
unset($_SESSION['k'])	Removes one value
session_destroy()	Ends the session entirely

A cookie, by contrast, stores the data in the browser itself with setcookie(), so it survives a server restart but can be read and edited by the user. Sensitive state belongs in the session, not in a cookie.

6. The Post/Redirect/Get Pattern

After a successful POST, the script should send a Location header and exit. This replaces the POST in the browser history with a GET, so refreshing the result page does not resubmit the form.

Course Project Implementation:

Course-Project Mapping: Not available in the course project. LipiSutra is a client-rendered React application with no server-side language: there is no .php file anywhere in the repository, no Apache configuration and no server-side session. The nearest equivalent is src/App.jsx, which persists the chosen theme in localStorage - but that is browser storage, not a server session, so it cannot be submitted for this assignment. A complete PHP application is therefore supplied below, implementing the LipiSutra historian login and manuscript-submission screens in PHP.

Requirement	In project?	Nearest equivalent
Server-side form handling	No	- (all handling is client-side)
Server-side validation	No	- (no validation layer at all)
String manipulation	Partially (JS)	src/services/tts.js strips XML tags with replace()
Session management	No (client only)	src/App.jsx localStorage.setItem("themePreference", ...)

Program:

File 1: login.php - session creation and a login guard

```
<?php
session_start();                                  // must precede any output

// a tiny demonstration user table; a real system reads a database
$USERS = [
    "prachi"  => ["pass" => "Lipi@2026", "name" => "Prachi Satbhai", "role" =>
"Historian"],
    "curator" => ["pass" => "Museum@26", "name" => "A. Curator",     "role" => "Museum
Admin"]
];

$error = "";

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $user = trim($_POST["username"] ?? "");
    $pass = $_POST["password"] ?? "";

    if ($user === "" || $pass === "") {
        $error = "Both username and password are required.";
    } elseif (!isset($USERS[$user]) || $USERS[$user]["pass"] !== $pass) {
        $error = "Invalid credentials.";
    } else {
        session_regenerate_id(true);              // prevent session fixation

        $_SESSION["username"]  = $user;
        $_SESSION["fullname"]  = $USERS[$user]["name"];
        $_SESSION["role"]      = $USERS[$user]["role"];
        $_SESSION["login_at"]  = date("d-m-Y H:i:s");
        $_SESSION["submitted"] = 0;               // a per-session counter

        header("Location: submit.php");           // Post / Redirect / Get
        exit;
    }
}
?>
<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8"><title>LipiSutra - Sign In</title></head>
<body style="font-family:Georgia,serif;background:#f5ede0;padding:24px">

  <h1>LipiSutra - Historian Sign In</h1>

  <?php if ($error !== ""): ?>
    <p style="color:#b00020"><?= htmlspecialchars($error) ?></p>
  <?php endif; ?>

  <form method="post" action="<?= htmlspecialchars($_SERVER["PHP_SELF"]) ?>">
    <label>Username</label><br>
    <input type="text" name="username"
           value="<?= htmlspecialchars($_POST["username"] ?? "") ?>"><br><br>

    <label>Password</label><br>
    <input type="password" name="password"><br><br>

    <button type="submit">Sign In</button>
  </form>

  <p style="font-size:13px;color:#5c3317">
    Demo accounts: prachi / Lipi@2026 &amp;nbsp;|&amp;nbsp; curator / Museum@26
  </p>
</body>
</html>
```

File 2: submit.php - protected page, validation and string manipulation

```
<?php
session_start();

// ---------- SESSION GUARD : only a signed-in user may open this page ----------
if (!isset($_SESSION["username"])) {
    header("Location: login.php");
    exit;
}

$errors = [];
$report = null;

// ---------- FORM HANDLING ----------
if ($_SERVER["REQUEST_METHOD"] === "POST") {

    // sanitize : trim and escape every incoming value
    function clean($value) {
        return htmlspecialchars(stripslashes(trim($value)), ENT_QUOTES, "UTF-8");
    }

    $title     = clean($_POST["title"]     ?? "");
    $accession = clean($_POST["accession"] ?? "");
    $email     = clean($_POST["email"]     ?? "");
    $year      = clean($_POST["year"]      ?? "");
    $script    = clean($_POST["script"]    ?? "");
    $transcript= clean($_POST["transcript"]?? "");

    // ---------- VALIDATION ----------
    if ($title === "") {
        $errors["title"] = "Title is required.";
    } elseif (mb_strlen($title) < 5 || mb_strlen($title) > 80) {
        $errors["title"] = "Title must be between 5 and 80 characters.";
    }

    if (!preg_match('/^LS-\d{4}$/', $accession)) {
        $errors["accession"] = "Accession number must look like LS-0147.";
    }

    if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
        $errors["email"] = "Enter a valid email address.";
    }

    $yearCheck = filter_var($year, FILTER_VALIDATE_INT,
                            ["options" => ["min_range" => -500, "max_range" =>
2026]]);
    if ($yearCheck === false) {
        $errors["year"] = "Year must be a whole number between -500 and 2026.";
    }

    $allowed = ["Modi", "Devanagari", "Brahmi", "Sharada"];
    if (!in_array($script, $allowed, true)) {
        $errors["script"] = "Choose one of the supported scripts.";
    }

    if (str_word_count($transcript) < 5) {
        $errors["transcript"] = "The transcript must contain at least 5 words.";
    }

    // ---------- STRING MANIPULATION on the accepted data ----------
    if (empty($errors)) {
        $_SESSION["submitted"] = ($_SESSION["submitted"] ?? 0) + 1;

        $slug = strtolower(str_replace(" ", "-", $title));          // str_replace +
strtolower
        $slug = preg_replace('/[^a-z0-9\-]/', '', $slug);
```

```
        $words   = str_word_count($transcript);                     // word count
        $chars   = mb_strlen($transcript);                          // multibyte-safe
length
        $preview = mb_substr($transcript, 0, 60);                   // substring
        if ($chars > 60) { $preview .= "..."; }

        $tokens  = explode(" ", $transcript);                       // string -> array
        $first5  = implode(" ", array_slice($tokens, 0, 5));        // array -> string

        $era = ($yearCheck < 0) ? abs($yearCheck) . " BCE" : $yearCheck . " CE";

        $report = [
            "Normalised title"   => ucwords(strtolower($title)),    // ucwords
            "URL slug"           => $slug,
            "Accession (upper)"  => strtoupper($accession),         // strtoupper
            "Submitted by"       => $_SESSION["fullname"] . " (" . $_SESSION["role"] .
")",
            "Email domain"       => substr($email, strpos($email, "@") + 1),  //
strpos+substr
            "Script"             => $script,
            "Era"                => $era,
            "Transcript words"   => $words,
            "Transcript chars"   => $chars,
            "First five words"   => $first5,
            "Preview"            => $preview,
            "Reversed accession" => strrev($accession),             // strrev
            "Submission number"  => $_SESSION["submitted"] . " this session"
        ];
    }
}
?>
<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8"><title>LipiSutra - Submit Manuscript</title>
<style>
  body  { font-family:Georgia,serif; background:#f5ede0; color:#1a0800;
padding:24px; }
  .err  { color:#b00020; font-size:13px; }
  table { border-collapse:collapse; margin-top:16px; }
  td,th { border:1px solid #8B4513; padding:6px 12px; font-size:14px; text-align:left;
}
  input,select,textarea { padding:6px; width:320px; }
</style></head>
<body>

  <!-- ---------- SESSION DATA READ BACK ---------- -->
  <p>
    Signed in as <b><?= htmlspecialchars($_SESSION["fullname"]) ?></b>
    (<?= htmlspecialchars($_SESSION["role"]) ?>) since
    <?= htmlspecialchars($_SESSION["login_at"]) ?>.
     <a href="logout.php">Sign out</a>
  </p>

  <h1>Submit a Manuscript</h1>

  <form method="post" action="<?= htmlspecialchars($_SERVER["PHP_SELF"]) ?>">

    <label>Title</label><br>
    <!-- STICKY FORM : the typed value survives a failed submit -->
    <input type="text" name="title"
           value="<?= htmlspecialchars($_POST["title"] ?? "") ?>">
    <span class="err"><?= $errors["title"] ?? "" ?></span><br><br>

    <label>Accession Number</label><br>
    <input type="text" name="accession" placeholder="LS-0147"
           value="<?= htmlspecialchars($_POST["accession"] ?? "") ?>">
    <span class="err"><?= $errors["accession"] ?? "" ?></span><br><br>

    <label>Email</label><br>
```

```
    <input type="text" name="email"
           value="<?= htmlspecialchars($_POST["email"] ?? "") ?>">
    <span class="err"><?= $errors["email"] ?? "" ?></span><br><br>

    <label>Estimated Year (negative for BCE)</label><br>
    <input type="text" name="year"
           value="<?= htmlspecialchars($_POST["year"] ?? "") ?>">
    <span class="err"><?= $errors["year"] ?? "" ?></span><br><br>

    <label>Script</label><br>
    <select name="script">
      <option value="">-- select --</option>
      <?php foreach (["Modi","Devanagari","Brahmi","Sharada"] as $s): ?>
        <option <?= (($_POST["script"] ?? "") === $s) ? "selected" : "" ?>><?= $s ?
></option>
      <?php endforeach; ?>
    </select>
    <span class="err"><?= $errors["script"] ?? "" ?></span><br><br>

    <label>Transcript</label><br>
    <textarea name="transcript" rows="4"><?= htmlspecialchars($_POST["transcript"] ??
"") ?></textarea>
    <span class="err"><?= $errors["transcript"] ?? "" ?></span><br><br>

    <button type="submit">Submit</button>
  </form>

  <?php if ($report !== null): ?>
    <h2>Accepted - Derived Report</h2>
    <table>
      <?php foreach ($report as $key => $value): ?>
        <tr><th><?= htmlspecialchars($key) ?></th>
            <td><?= htmlspecialchars((string)$value) ?></td></tr>
      <?php endforeach; ?>
    </table>
  <?php elseif (!empty($errors)): ?>
    <p class="err"><?= count($errors) ?> field(s) need attention.
       Your entries have been preserved.</p>
  <?php endif; ?>

</body>
</html>
```

File 3: logout.php - destroying the session

```
<?php
session_start();

$_SESSION = [];                       // clear the in-memory data

// delete the session cookie from the browser as well
if (ini_get("session.use_cookies")) {
    $p = session_get_cookie_params();
    setcookie(session_name(), "", time() - 42000,
              $p["path"], $p["domain"], $p["secure"], $p["httponly"]);
}

session_destroy();                    // remove the file on the server

header("Location: login.php");
exit;
```

Steps to run

```
1. Install XAMPP and start the Apache service.
2. Copy login.php, submit.php and logout.php into C:\xampp\htdocs\lipisutra\
   (on Linux: /opt/lampp/htdocs/lipisutra/).
```

```
3. Open http://localhost/lipisutra/login.php in the browser.
4. Sign in with prachi / Lipi@2026.
5. Submit the manuscript form, first with invalid values, then with valid ones.
6. Click "Sign out" and try to open submit.php directly to test the session guard.
```

Output:

- Opening submit.php without signing in redirects straight back to login.php, because the session guard finds no $_SESSION['username'].
- Signing in with the wrong password prints "Invalid credentials." and keeps the typed username in the field.
- After a successful sign-in the browser is redirected to submit.php, which greets the user by name and role and shows the login time - all read from the session.
- Submitting "LS-147", a bad email and a 3-word transcript shows four inline error messages and the notice "4 field(s) need attention. Your entries have been preserved." with every typed value still present.
- Submitting valid data prints the derived report: the normalised title "Modi Land Grant Of 1712", the slug "modi-land-grant-of-1712", the accession in upper case, the email domain, the era rendered as "1712 CE", the word and character counts, the first five words, a 60-character preview, the reversed accession number and "Submission number 1 this session".
- Submitting a second manuscript shows "Submission number 2 this session", proving the counter is held on the server between requests.
- Clicking Sign out destroys the session; opening submit.php again redirects to the login page and the counter restarts at 1 after the next sign-in.

Conclusion:

In this assignment, we developed a PHP application demonstrating server-side form handling with $_POST, validation and sanitization with trim, htmlspecialchars, filter_var and preg_match, the sticky-form pattern, a wide range of string functions including ucwords, str_replace, mb_substr, strpos, explode, implode, str_word_count and strrev, and session management with session_start, session_regenerate_id, a login guard and session_destroy. We also established that the LipiSutra course project contains no server-side code at all, so this application was written specifically for the assignment.

References:

8. PHP Manual - Language Reference, https://www.php.net/manual/en/langref.php
9. PHP Manual - Session Handling, https://www.php.net/manual/en/book.session.php
10. W3Schools - PHP Tutorial, https://www.w3schools.com/php/
11. PHP Manual - filter_var, https://www.php.net/manual/en/function.filter-var.php
12. PHP Manual - String Functions, https://www.php.net/manual/en/ref.strings.php